

Multi bunch feedback application software

Alun Morgan

June 2026

Contents

1	Overview	2
1.1	Conceptual descriptions	2
1.2	Common application behaviour	3
1.3	Frontend Optimisation	3
1.3.1	Measurements	5
2	System setup	7
2.1	Code location	7
2.2	MBF frontend open loop setup	7
2.2.1	DORIS phase	7
2.2.2	Frontend Overview	10
2.2.3	Frontend tuning	12
2.3	MBF closed loop setup	19
2.3.1	Initial measurement	19
2.3.2	Assessment	20
2.3.3	How to tune the loop delay	20
2.4	MBF measurements	23
2.4.1	Code to run	23
2.4.2	Emittance control	25
2.4.3	Bunch motion	25
2.4.4	Bunch cleaning	25
2.4.5	Chromaticity	25
2.4.6	Tune scan over modes	25
3	MBF Archival retrieval	26
3.1	Introduction	26
3.2	Getting the latest data	26
3.3	Getting any historic data	31
3.3.1	Growdamp	32
3.3.2	Modescan	35
3.3.3	Spectra	35

Chapter 1

Overview

1.1 Conceptual descriptions

The hardware for multi bunch feedback has many different possible configurations. Broadly speaking a feedback system has a frontend which takes in the signals from the pickups, and does the initial analogue signal conditioning. These signals are then fed into digital systems for each of the three planes (x,y,s). These digital systems calculate the feedback required and send signals to analogue drive electronics in order to excite the beam in the given plane. This is summarised in figure 1.1

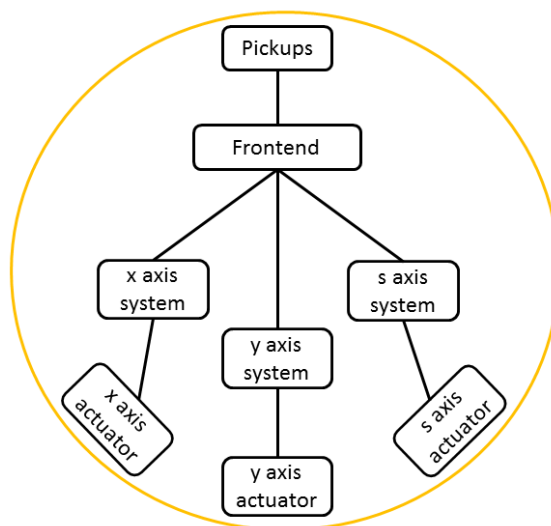


Figure 1.1: System overview

In order to make such a system more easily usable for a wide range of system users, a suite of task specific applications has been developed. The aim of the suite is to abstract away the implementation detail of the hardware and thus allow the users to put their attention on type of measurement they want to do. This abstraction also aids in the development of bespoke measurements for specific investigations.

1.2 Common application behaviour

All these codes save the relevant data and graphs to a file structure based on the date, and given unique names based on the date and time the application was run. The root path for this structure is defined in `mbf_system_config.m`, as is the harmonic number of the machine under study. In general all machine/deployment parameters are set in this file. The rest of the codebase is written to adapt to what is set here.

In addition to the system data from the multi bunch hardware, more general machine parameters are also captured to enable machine studies (e.g current dependence). This additional data is set in `machine_environment.m`.

1.3 Frontend Optimisation

The frontend as an analogue RF system which allows control of two aspects. First, adjusting the gain of the signals so that we can use the appropriate amount of the ADC input range. Second, in order to work optimally the signals need to be matched in timing/phase with the beam phase (not the RF phase) and be tuned so that the sampling point of the ADC coincides with the maximum signal from each bunch.

The input level is decided by which operational condition you want to optimise for. Common options are injection, a uniform fill, or camshaft / hybrid mode where some buckets are filled with more charge. ADCs usually have a range over which they are linear and then a range beyond which where they have a non-linear response, eventually saturating. It may be that you want normal operation to be in the linear region but you can afford the injection to be in the non-linear region as long as it is below saturation. The general approach to setting the frontend gains is that you put the machine in the desired operational condition and adjust the frontend gains to bring the signal level into the correct region of the ADC range () more details on how to do this are in section ??.

The first timing requirement of aligning to beam phase is done by phase shifting the input signal. At its most basic this comprises of a single phase shifter for each channel which both allows each channel to be adjusted to follow changes in beam phase, and to allow the axis to be tuned in order to account for differences in the phase delay of that channel.

The practical implementation will depend on the details of your frontend hardware. However is often realised as a single LO phase shifter which is common to all axes and handles the beam phase adjustments. meanwhile each channel has an individual additional phase shifter to tune out the systemic differences in phase delay between the channels. Such a setup is shown in figure 1.2. Another variation is replacing the LO phase shifter with an automatic beam phase following system such as a DORIS ((Delayed Orbit Reference Improvement Scheme) unit which locks the MBF to a fixed relationship to beam phase rather than the RF phase. DORIS will lock if there is more than 3 mA in the storage ring.

The second timing requirement is done by adjusting the sampling clock until a maximum signal is obtained with a minimum signal leakage to adjacent RF buckets. See figure 2.9 for an example.

Note: The Libera bunch by bunch frontend seems to get into problems when scanning the LO phase shifter. It is probably best not to scan the LO phase if you have that frontend.

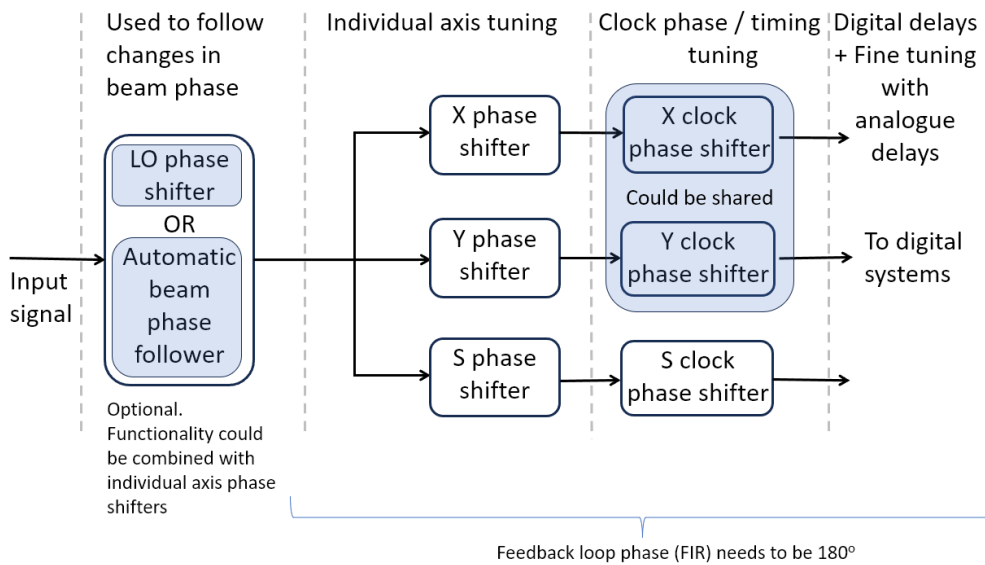


Figure 1.2: phase shifters

1.3.1 Measurements

DORIS scans

Scan the phase of the DORIS unit to identify an optimum target value for the feedback loop.

Basic usage:

```
DORIS_target_phase_scan(<bunch>)
```

Frontend scans

Scan the relevant phase shifter or clock delay while monitoring the measure signal amplitude. The resulting graphs will allow appropriate working points to be set. **ADD SOMETHING ABOUT AMPLITUDE SCANS**

Basic usage:

```
BBBFE_full_frontend_checkout
```

Mode scans

A mode scan is a sequential excitation of the harmonics of the tune frequency. This is mainly a system check in order to identify any performance limitations or degradation.

Basic usage:

```
modescan_all('x')
```

Spectra

This code captures the requested number of turns from the requested axis and returns the FFT of the centroid motion of each bunch. This is useful for identifying tunes, tune leakage from one axis to another, also for classifying instabilities. **EXTEND ANALYSIS TO EXTRACT TUNE LEAKAGE EXPLICITLY**

Basic usage:

```
mbf_spectrum_all('x')
```

Growdamp

The measurement comprises a series of measurements at set harmonics of the tune frequency capturing the evolution of bunch both with and without feedback active. This can be with or without excitation. The former is the standard measurement of response to a given excitation. The latter is useful to find which modes the machine naturally excites.

The excitation is usually on the tune but can be set to follow a sideband or even a fixed point in tune space. Tune excitation is used to investigate the behaviour of the bunch centroid while sideband excitation is used to investigate then effect on the bunch size

Basic usage:

```
growdamp_all('x')
```

Emittance control

This uses one of the oscillators to excite the selected bunches at one of the sideband frequencies. This has the effect of blowing up the beam. Basic usage:

Bunch motion capture

The three systems are setup to be ready to capture on the next external trigger. Once that trigger arrives, the three systems simultaneously capture bunch centroid motion. This is useful for capturing injection transients. Basic usage:

Chapter 2

System setup

2.1 Code location

The code is released under an MIT licence. In order to use it please grab it from the [code repository](#) or use the following command on the system command line in order to copy it to your current location:

```
git clone https://github.com/alunmorgan/Multi-bunch-feedback-applications.git
```

and add the resulting folder to your Matlab path. If you find odd behaviour check that you do not have any other code in the Matlab path as you could have name clashes.

If you have already cloned the repository navigate to the folder and run

```
git pull
```

This will ensure you have the most up to date version of the code.

This and related documentation can be found in the [documentation folder in the repository](#)

2.2 MBF frontend open loop setup

Machine settings

Insertion devices (IDs) need to be set to injection gaps.

2.2.1 DORIS phase

Machine settings

Inject a full fill 10 mA.

DORIS phase scan

Before running the code to scan the DORIS phase, select a single bunch to look at. Make sure the single bunch location has charge in it.

The following code will scan the phase of the DORIS unit to identify an optimum value. It will select bucket 400 as the target single bunch location.

```
>> DORIS_target_phase_scan(400)
```

ADC		DAC	
Phase:	74.98 deg	Phase:	-2.96 deg
Level:	0.471	Control:	-2.87
Target:	75.00	I/Q:	0.699 -0.036
Error:	0.010 deg		
I/Q:	0.122 0.455		

System Status		Control	
DORIS Clock Alignment System rev 2.0		P:	0.00000
Oct 14 2019		I:	0.10000
Watchdog: On		D:	0.00000
Connection:	Connected	Navg:	16
Signals:	Ok	Threshold:	0.04980
Lock:	Locked		
Feedback:	Closed Loop		
Temperature:	22.3 degC		
Up time:	2672773 sec 30 days 22:26:13		
Lock time:	115872 sec 1 days 08:11:12		

Figure 2.1: DORIS settings

If you want to check that the settings are changing as expected then look at the screen in figure 2.1 (Diagnostics → SR → DORIS).

Figure 2.2 shows an example of the output.

The optimum value is the phase where one gets the maximum signal of the excited bunch. Adjust the DORIS target phase to this phase value.

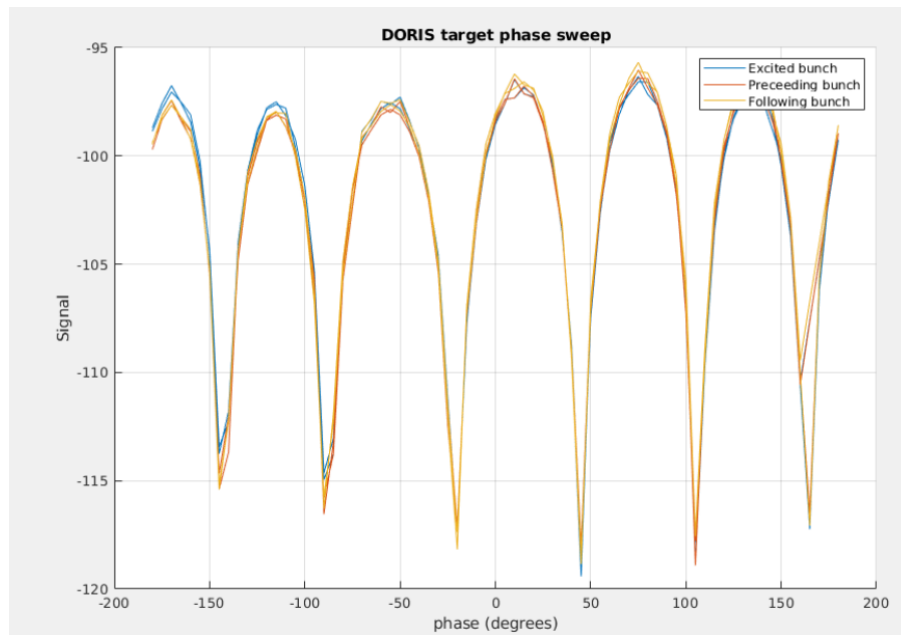


Figure 2.2: In this case 75° looks to be a reasonable working point.

2.2.2 Frontend Overview

As both the amplification (gain) and timing (phase) adjustments can change the final input signal amplitude to the ADC it may be required to revisit one adjustment after a different one has been completed. For example you check and optimise the gains, then optimise the system phase and clock phase. This further increases the signal at the ADC and now required the gain to be reduced in order to stay in the linear range. In the front end there are two phases (delays) we are interested in optimising. The system phase which determines how accurately our detectors are aligned with the target bunch, and the clock phase which determines how close to the peak of the signal we are sampling.

Each axis in the frontend has individual gain control (fig 2.3)

Bunch by Bunch Front End	
Status	
Fan speed	4200 rpm
Internal temperature	31.2 deg C
Alarm status	NO
Uptime	0033:11:13
Voltages	
3.3V supply voltage	3.26 V
5V supply voltage	4.99 V
-5V supply voltage	-4.52 V
8V supply voltage	7.95 V
12V supply voltage	10.91 V
Status Control	
LCD sleep mode time	20
LCD sleep brightness	10
LCD normal brightness	100
Target fan speed	4000
Shutdown temperature	45
MC Phase Shifts	
clk1 phase shift	3
clk2 phase shift	0
clk3 phase shift	-80
clk4 phase shift	0
Input Levels	
X input level	-26 36 dB
Y input level	-26 36 dB
I input level	-10 20 dB
LO Phase Shifts	
LO phase shift	0
X phase offset	-40
Y phase offset	-40
IT phase offset	-160
IL phase offset	-10
Device Identifier IT L-BBFE	
EXIT	

Figure 2.3: Frontend settings: Gains in input levels section. System phases in LO Phase shifts section X=X, Y=Y S=IT. Clock phases in the MC Phase Shifts section X and Y=clk3, S=clk4

The ADC graphs show fraction of full scale (1 = 100%). See figure 2.4

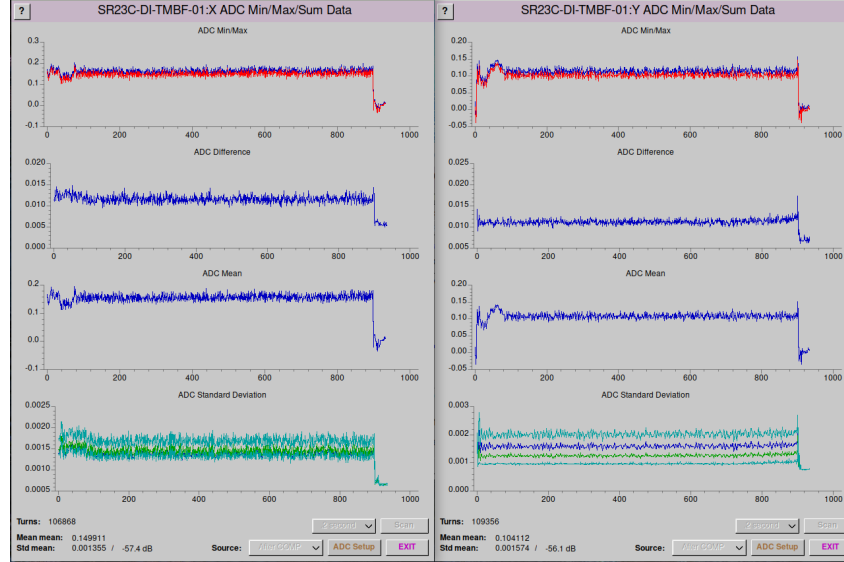


Figure 2.4: Example ADC graphs. In the case of a 900 bunch (out of 936) fill, the ADC maximum is $\approx 15\%$ for X and $\approx 10\%$ for Y when no instability is present.

2.2.3 Frontend tuning

Start with the frontend phases then move on to the frontend gains.

Machine settings

- Inject a single bunch into a selected RF bucket (0.2nC).
- if no sync press sync button as shown on figure 2.7
 - (Diagnostics → SR → TMBF-X/TMBF-Y/LMBF → System → Delays)

Alignment to timing system

- Compare the MBF ADC data and the fillpattern to the bunch number requested.
 - Look at the Bunch motion standard deviation.
 - * Diagnostics → SR → TMBF-X/TMBF-Y/LMBF

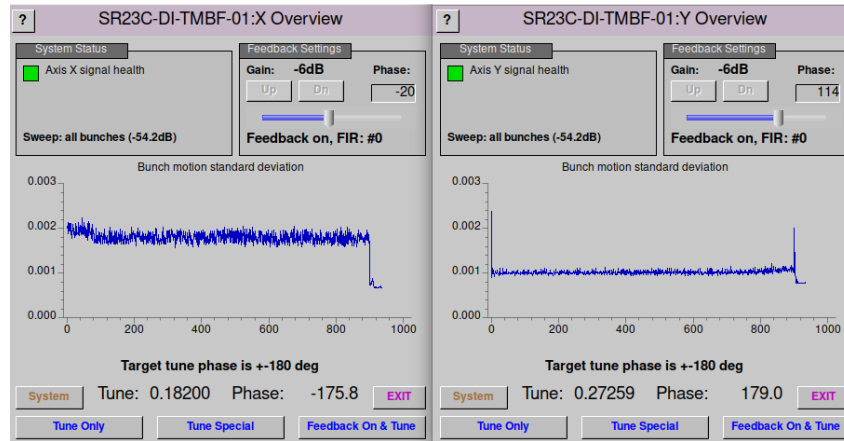


Figure 2.5: Bunch motion in normal operation **NEED TO ADD SINGLE BUNCH EXAMPLE**

- Look at fill pattern
 - * (Diagnostics → Photon counting system → fill)
- adjust MBF and fillpattern until they agree with the requested bunch number as given by the timing system.
 - adjust MBF bunch value as shown on figure 2.7

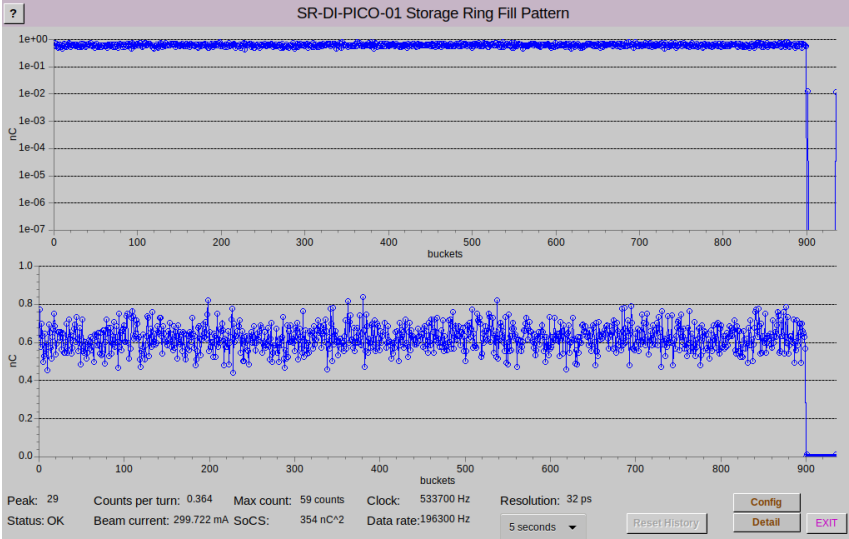


Figure 2.6: fill pattern in normal operation **NEED TO ADD SINGLE BUNCH EXAMPLE**

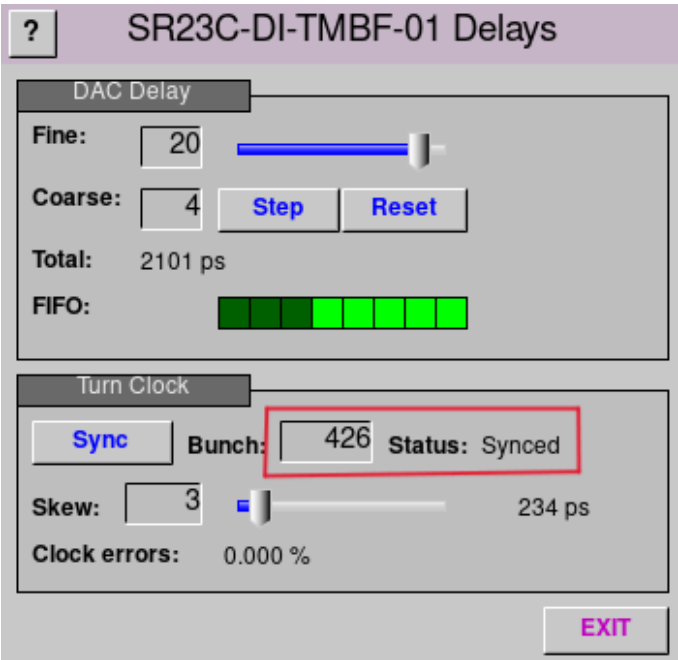


Figure 2.7: MBF delay screen

System phase scan

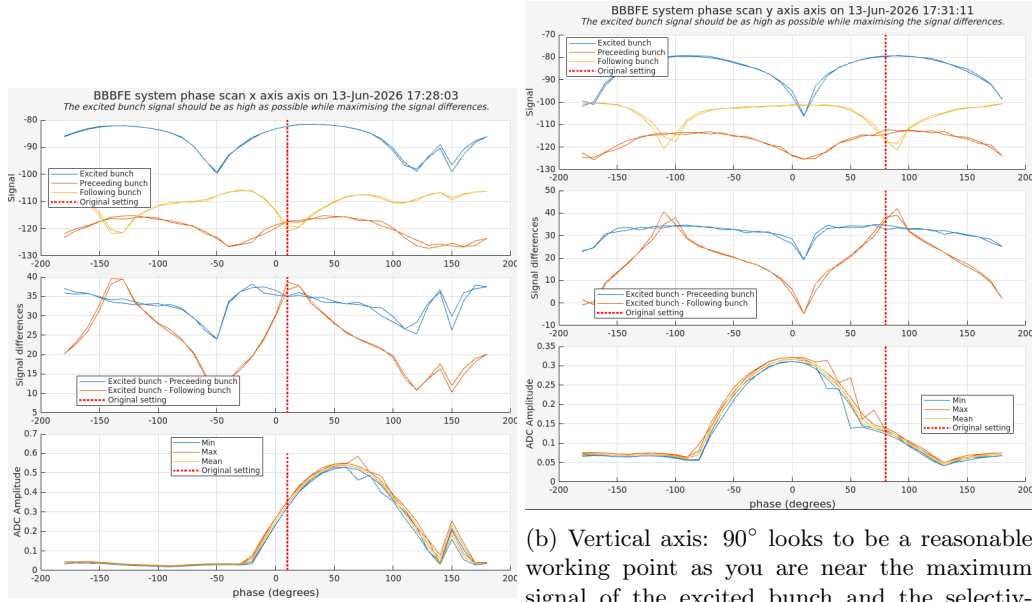
The following Matlab code will run a system phase scan on each axis sequentially. The graphs will be displayed, and the data is automatically stored.

```
>> BBBFE_system_phase_scan('x', single_bunch_location)
>> BBBFE_system_phase_scan('y', single_bunch_location)
>> BBBFE_system_phase_scan('s', single_bunch_location)
```

Figure 2.8 shows an example of the output. You are looking for the phase value with the largest gap between the excited bunch and each of the other bunches. But also fairly near the maximum excited bunch maximum amplitude.

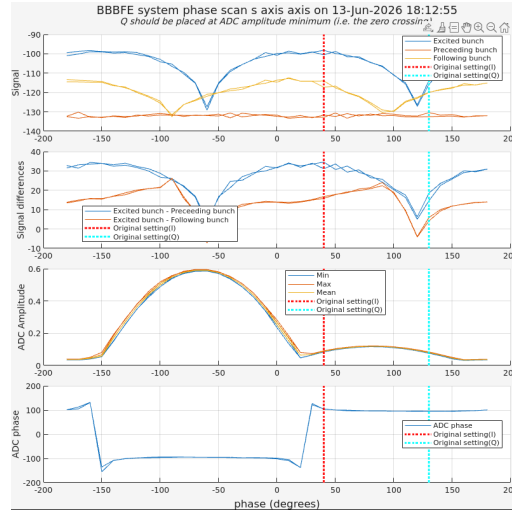
Note: If the graphs are significantly asymmetric re-run the phase scan to check.

Note: If there are missing data points it can indicate that the DORIS system has unlocked. In this case you may need to inject a little more charge into the target bucket.



(a) Horizontal axis: 10° looks to be a reasonable working point as you are trading of better selectivity for better signal to noise.

(b) Vertical axis: 90° looks to be a reasonable working point as you are near the maximum signal of the excited bunch and the selectivity for the preceding and following bunch's are also close to maximum. However you have lower ADC amplitude



(c) longitudinal axis: 20° looks to be a reasonable working point for Q as it is near the zero crossing of the ADC phase. I then has to be -70° to keep the quadrature condition.

Figure 2.8: Example system phase results

Clock phase scan

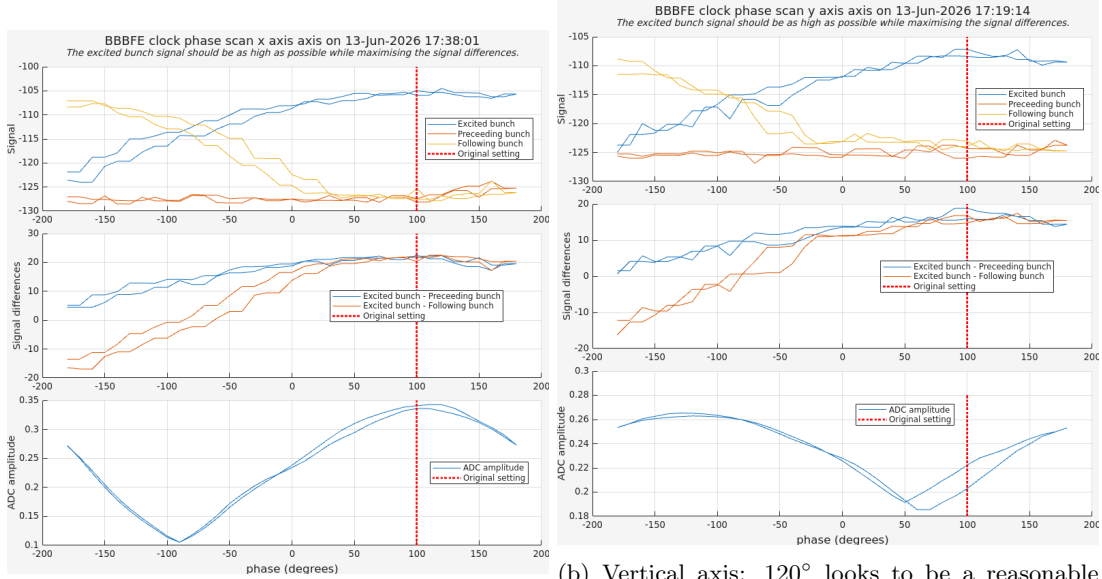
The following Matlab code will run a clock phase scan on each axis sequentially. The graphs will be displayed, and the data is automatically stored.

```
>> BBBFE_clock_phase_scan('x', single_bunch_location)
>> BBBFE_clock_phase_scan('y', single_bunch_location)
>> BBBFE_clock_phase_scan('s', single_bunch_location)
```

Figure 2.9 shows an example of the output. You use the same assessment as for the system phase scans.

Note: X and Y share a clock so you have to optimise both axes together.

Note: Check the bunch standard deviation graphs to check that the motion is still only in a single bucket. If it is split over two then you may have to choose an alternative working phase.



(a) Horizontal axis: 120° looks to be a reasonable working point. lose to maximum signal and equalising the trade off between the preceding and following bunches.

(b) Vertical axis: 120° looks to be a reasonable working point. As the clock for X and Y is common they need a solution which works for both. If they are significantly different you may have to go back and adjust the system phases.



(c) longitudinal axis: 160° looks to be a reasonable working point.

Figure 2.9: Example clock phase results

frontend gain scan

Machine settings

Put the machine in the desired condition (i.e. uniform fill)

Adjust the gain on each channel so that the level on the ADC is 10 – 15% with no instabilities present.

Once the system phase, clock phase and frontend gain are optimised then rerun all measurements to make sure that later adjustments have not disturbed earlier ones. If all is good then the initial tuning of the front end is complete.

The next step is to close the feedback loop.

2.3 MBF closed loop setup

Machine settings

- From an empty machine Inject to 50 mA into all bunches.
 - this should give sufficient signal while staying below the instability threshold
 - Check there is no activity on the standard deviation of motion graph (Figure 2.10)
- Correct tune values to nominal.
 - If you are struggling to get a reliable tune value try
 - * increasing the excitation (Figure 2.11a)
 - (Diagnostics → SR → TMBF-X/TMBF-Y/LMBF → System → Control)
 - * changing from fitted to maximum (Figure 2.11b)
 - (Diagnostics → Summary → Click tune value)
- Turn tune feedback off once tunes are nominal
- Correct orbit using fast orbit feedback (FOFB). This can be left on

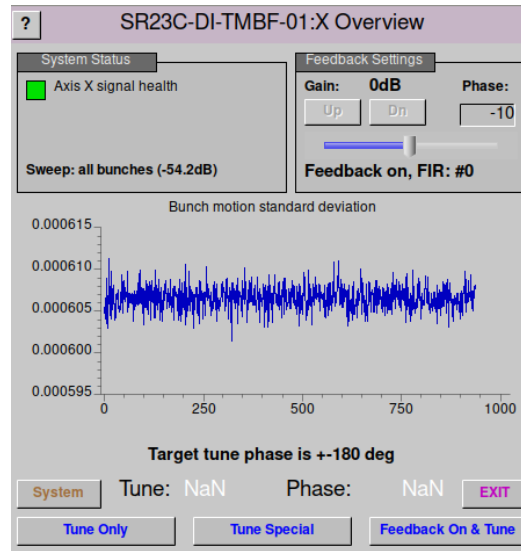


Figure 2.10: If you see distortion of the fill pattern or large amounts of jumping then you are probably in an unstable region. Drop the beam and refill to a lower current.

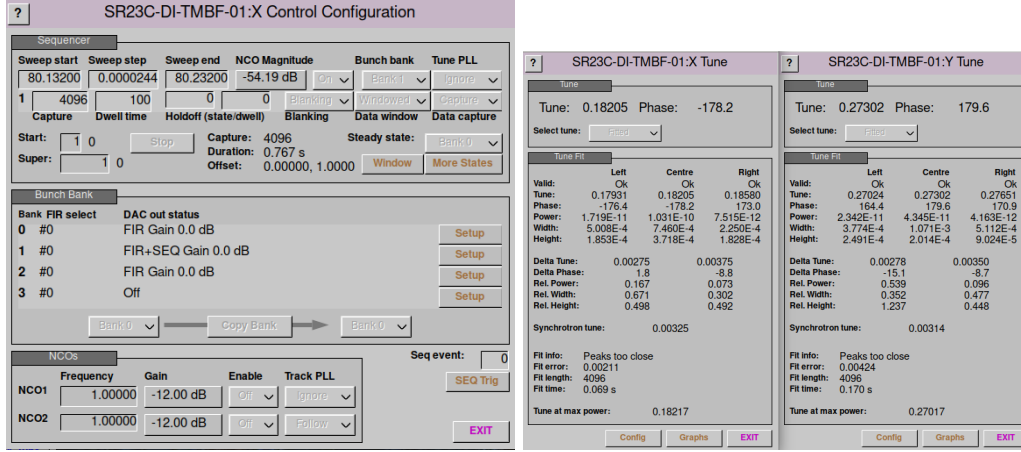
2.3.1 Initial measurement

The following Matlab code will run a scan across all the harmonics of the tune on each axis sequentially. The graphs will be displayed, and the data is automatically stored.

```
>> modscan_all('x')
>> modscan_all('y')
>> modscan_all('s')
```

2.3.2 Assessment

Use the modescan results to assess whether the closed loop delay is good enough.



(a) Tune setup

(b) Tune measurement settings

The amplitude should be fairly symmetric in the modes axis. If there is a significant change in the lowest modes only then check you have filled all bunches. The fewer bunches you fill the larger the distortion of the lower mode response becomes.

The phase variation should be much less than $\pm 90^\circ$ and the extents should be balanced between going above and below zero.

By taking a linear fit along the modes one can estimate the required electrical delay needing to be added or removed from the loop. Positive gradient means length must be added while negative gradient means length must be removed.

This error in the electrical length / delay / phase can be fixed in the following ways.

- First see if a DAC delay can fix it (using the control panel shown in figure 2.13). This is only available in 2ns steps.
- Otherwise, we must change cable lengths. The easiest place to do this is at the output port of the output amplifier. As shown in figure ??

Once the loop is closed for all modes the FIR phase can be adjusted to maximise the feedback efficacy

2.3.3 How to tune the loop delay

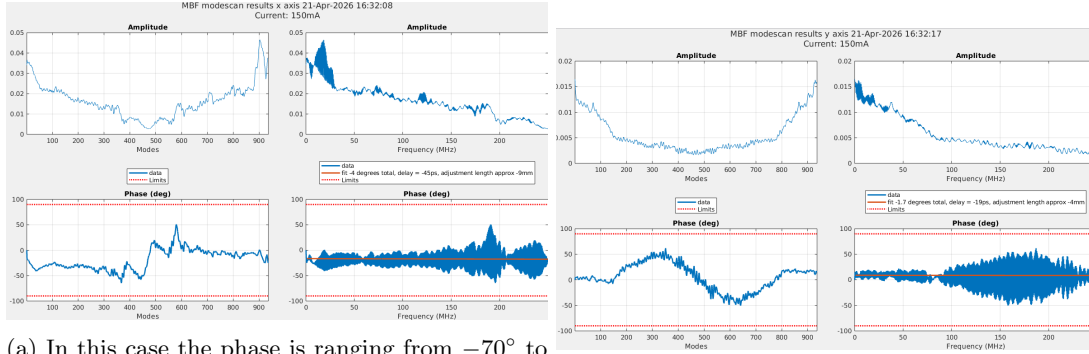
The aim of this exercise is to get the extents of the phase trace comfortably fit within the $\pm 90^\circ$ limits (similar to the result show in figure 2.12b).

Rerun the modescan code after each change.

Note: The spec for the phase variation of the output amplifier alone is ± 20 .

Tuning the DAC timings

If you see a large ($> 2ns$) change over all the modes then the first thing to look at is the DAC timings. These are independent for each axis.



(a) In this case the phase is ranging from -70° to 50° and there is almost no residual gradient along the modes. Although any further increase in this range would be concerning and would warrant further investigation.

(b) The phase is ranging from -50° to 50° and there is almost no residual gradient along the modes indicating that the loop delay is close to the required value.

Tuning the DAC delay

This allows additional phase (delay) to be added to the closed loop.

Note: The DAC delay is shared between X and Y axes so it may be required to retune both.

If delay needs to be reduced then the bunch setting can be reduced by one to reduce the overall delay by 2ns. Additional delay can be then added to reach the target delay. Use DAC delay first to get as close as possible to the target performance. Any remaining adjustment will have to be done with physical cable length changes. However one should check the bunch position with respect to the timing system once the loop has been successfully closed for all modes.

Tuning the cable length

Modify the cable length until the extents of the phase trace comfortably fit within the limits. A

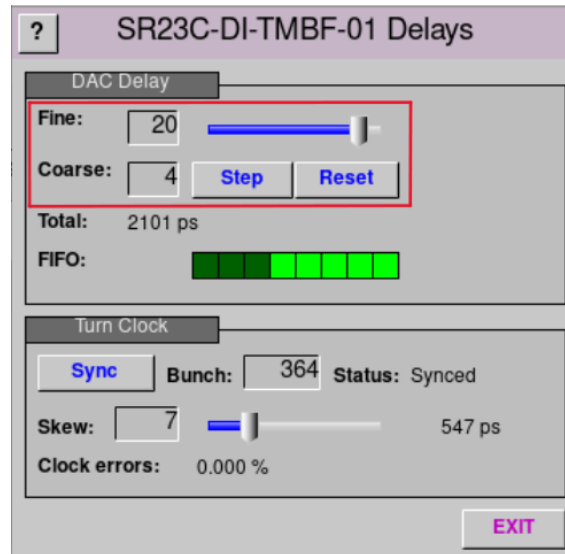


Figure 2.13: DAC delay control screen

large phase variation even when the overall gradient is close to zero indicates that a component is

failing – probably the output amplifier. So, a hardware change and a retuning of the frontend is required.

Tuning the FIR phase

The overall phase of the feedback loop should be 180 degrees. So the last step is to change the FIR phase setting so the readback value is close to (± 20) 180.

At this point the full loop delay should be optimised

2.4 MBF measurements

Machine settings

- Inject to 30 - 60mA $\geq$ 900 bunches (Ideally it should be 936 bunches and below the instability threshold)
- Set FIR gains on the MBF feedback to nominal values

2.4.1 Code to run

The following Matlab code will run modescan, growdamp and spectra measurements for each axis sequentially. The data is stored automatically

```
>> mbf_startup_tests
```

The following explains how to run each separate standard measurement. However there are some common inputs.

- Required
 - `'mbf_axis', 'x' or 'y' or 's'`.
The axis being measured.
- Optional
 - `'plotting', 'yes' or 'no'`. Default is yes
Determines if the plotting is displayed. The raw data is saved in either case.
 - `'additional_save_location', string` representing a filepath. Default is NaN
Saves the data in the requested location in addition to the standard location.
 - `'auto_setup', 'yes' or 'no'`. Default is yes
This determines whether to use the main scripts to reset the system to a known state. However this is dangerous if the machine is not in an intrinsically stable condition.

Growdamp

The top level code to run a growdamp measurement for each axis is;

```
>> growdamp_all('x')
>> growdamp_all('y')
>> growdamp_all('s')
```

The parameters for the length of excitation and capture are set in `mbf_growdamp_config.m`. The `mbf_growdamp_setup` code sets up the hardware for the correct type of measurement.

`mbf_growdamp_capture` triggers the measurement, captures the data and stores it in the file structure defined in `mbf_system_config`. If plotting is selected the code shows the last saved dataset.

Additional optional settings

- `excitation(str)`: sets whether the measurement uses an excitation to drive a resonance or if the measurement just uses the natural beam coupling. Defaults to 'yes'
- `excitation_location(str)`: The location of the excitation in tune space. The default value is 'tune'.

- `excitation_manual(float)`: if `excitation_location` is set to 'manual' then this defines the location of the excitation in tune space.
- `pll_tracking (str)`: Sets if the excitation frequency follows the tune jitter. Default is no
- `pll_bunches (float)`: bunches the pll is active on.
- `pll_guard_bunches (float)`: the number of bunches surrounding the pll bunches for which feedback is switched off.
- `bunch_monitor(list(int))`: This determines which bunches are monitored during the measurement. Defaults to an array of ones which means all bunches.
- `capture_full_bunch_motion(str)`: Captures the centroid motion for the duration of the measurement. Defaults to no as this is a lot of data.

Modescan

The top level code to run a modescan measurement for each axis is;

```
>> modescan_all('x')
>> modescan_all('y')
>> modescan_all('s')
```

The `mbf_modescan_setup` code sets up the hardware for the correct type of measurement. `mbf_modescan_capture` triggers the measurement, captures the data and stores it in the file structure defined in `mbf_system_config`.

Additional optional settings

- `n_repeats(int)`: How many times to repeat each measurement point.
- `dwel (int)`: How many turns to stay at each measurement point.
- `excitation_gain(float)`: level of the excitation.

Spectra

The top level code to run a modescan measurement for each axis is;

```
>> mbf_spectrum_all('x')
>> mbf_spectrum_all('y')
>> mbf_spectrum_all('s')
```

The `mbf_spectrum_setup` code sets up the hardware for the correct type of measurement. `mbf_spectrum_capture` triggers the measurement, captures the data and stores it in the file structure defined in `mbf_system_config`.

Additional optional settings

- `n_turns (int)`: number of turns to capture. Default is 1000.
- `repeat (int)`: Number of repeat points (improves the power resolution). Default is 1.

The MBF system is also used as an instrument for investigating the accelerator behaviour and the behaviour of the beam under different conditions. It can also be used as a driver for certain beam behaviours. These codes tend to be less well tested as they are often only used for particular investigations.

2.4.2 Emittance control

2.4.3 Bunch motion

`mbf_bunch_motion_setup` sets up all three systems to use the appropriate triggers.

`mbf_bunch_motion_capture` arms the systems, and once triggered by an external trigger, will capture and store the data.

2.4.4 Bunch cleaning

2.4.5 Chromaticity

Uses the scan of tune and sidebands to find the chromaticity of the machine.

2.4.6 Tune scan over modes

Scan to allow optimisation of the tune measurement.

Chapter 3

MBF Archival retrieval

3.1 Introduction

All the MBF codes store their data in a common location with programmatically constructed names. Therefore, it is possible to extract and compare this data.

3.2 Getting the latest data

The simplest thing is to look at the last results taken. To do this simply run the command below on the Matlab command line.

```
>> visualise_latest_mbf_results
```

This will return analysed results for all the standard measurements.

There are optional flags to modify the standard behaviour.

- `'save_graphs', 'yes', 'out_path', [your desired location])`

Save PNGs of the graphs at a location of your choosing.

- `'measurements', {'Growdamp', 'Modescan', 'Spectrum', 'system_phase_scan', ...
'clock_phase_scan', 'DORIS_phase_scan'}`

Select only the measurements you are interested in.

- `'axes', {'x', 'y', 's'}`

Select only the axes you are interested in.

Figure 3.1 is an example of the output for Growdamp, figure 3.2 of Modescan measurements and figure 3.3 of spectral measurements.

The spectra graphs show the residual motion after all feedbacks have been applied.

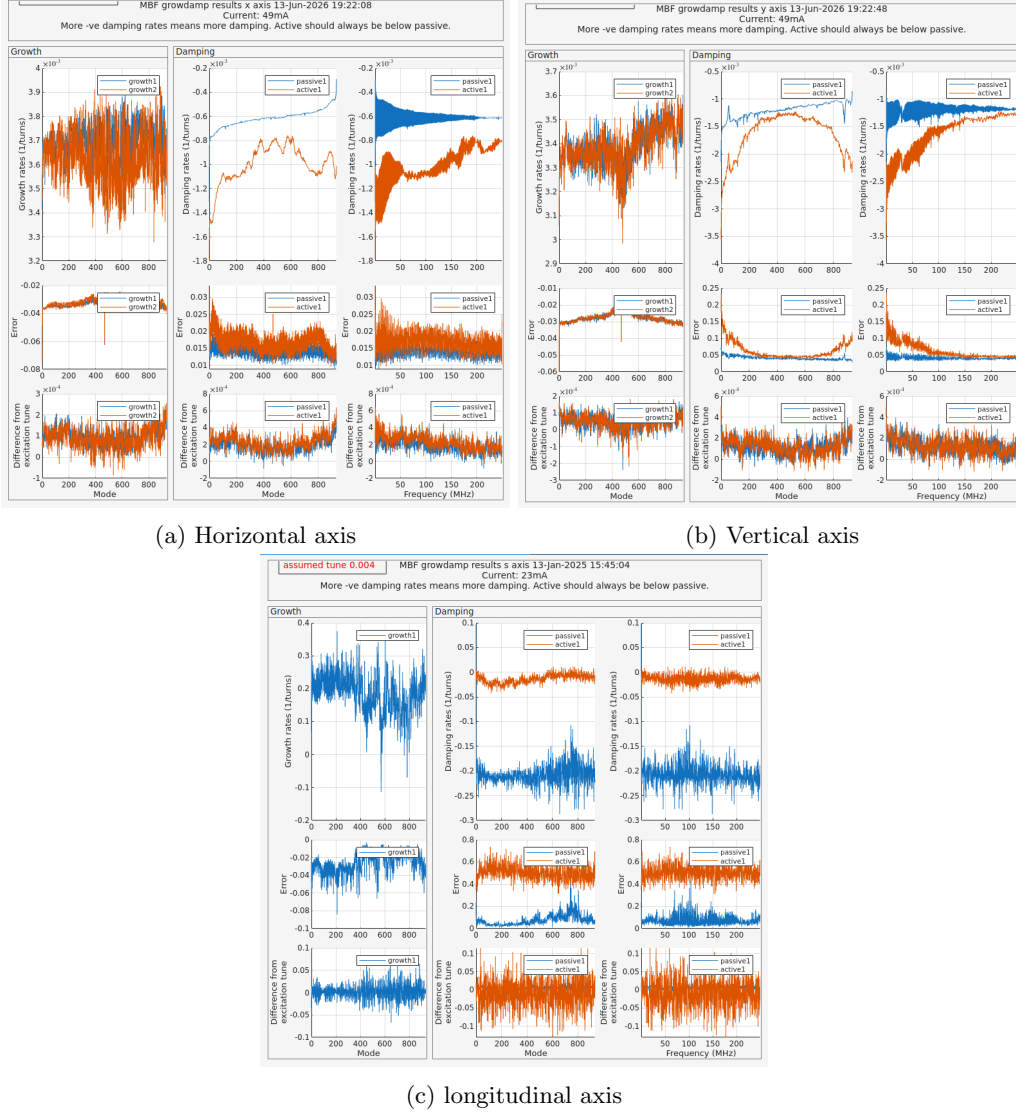


Figure 3.1: Example Growdamp results

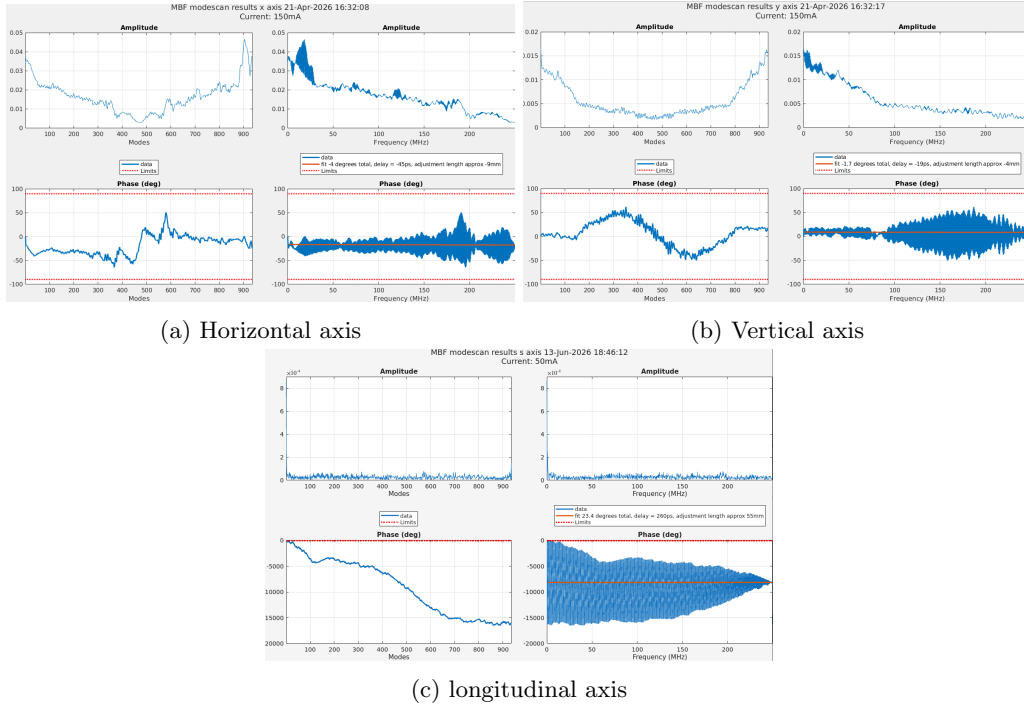


Figure 3.2: Example modescan results

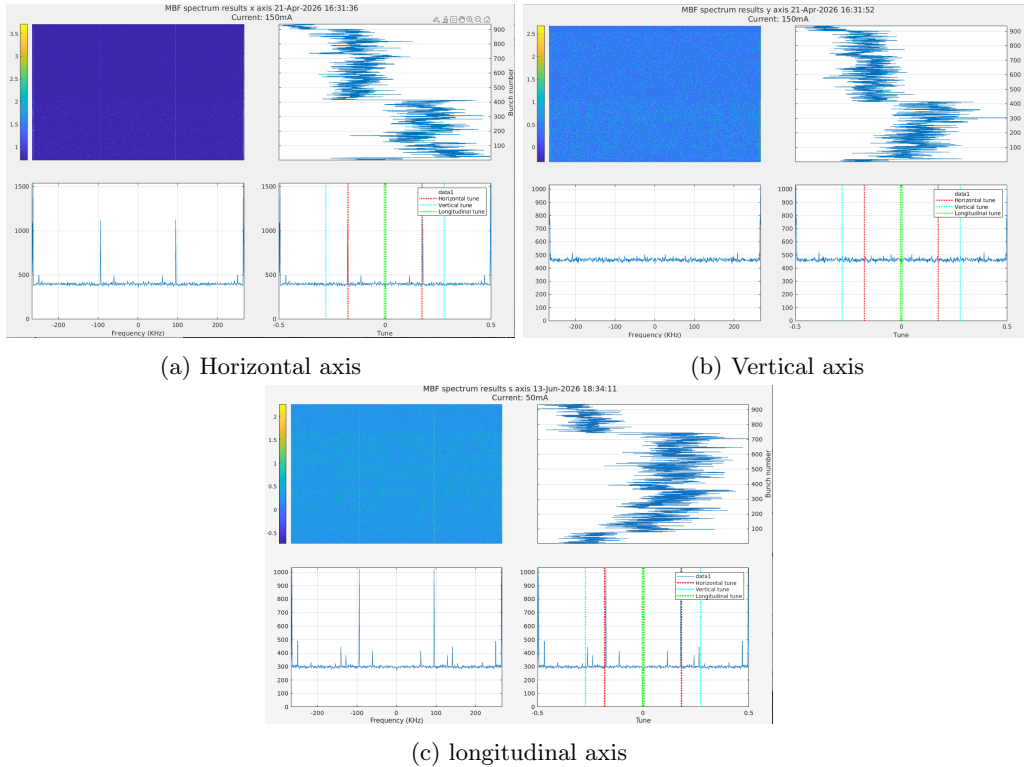
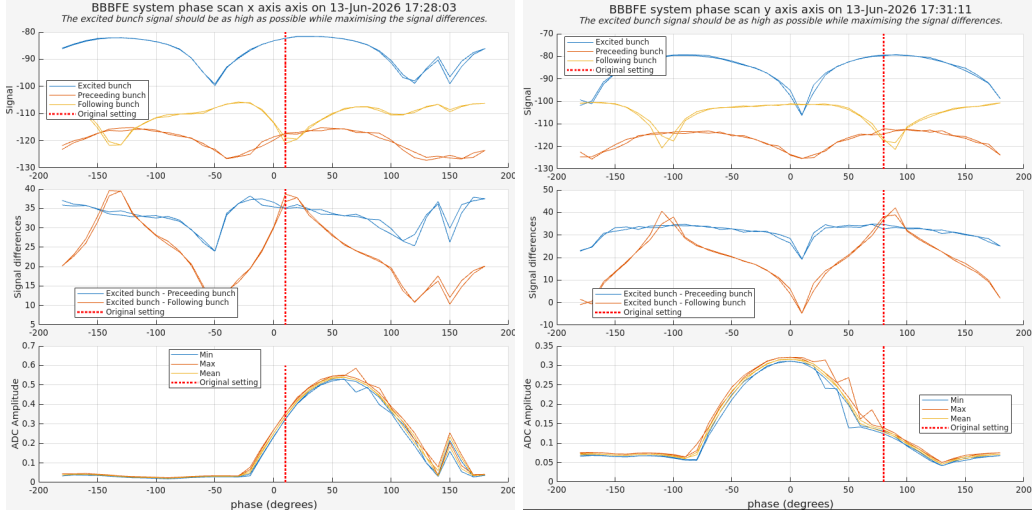
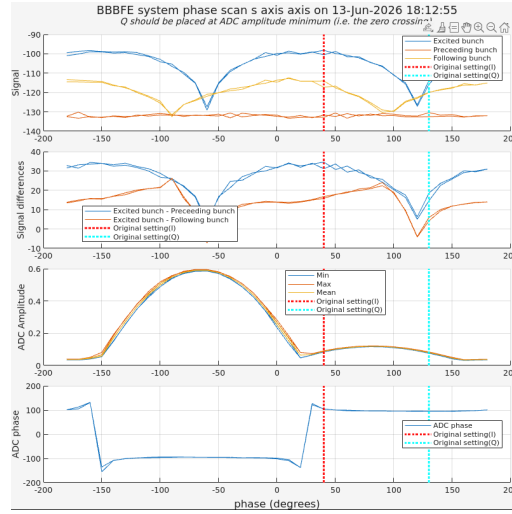


Figure 3.3: Example spectrum results- PLACEHOLDER



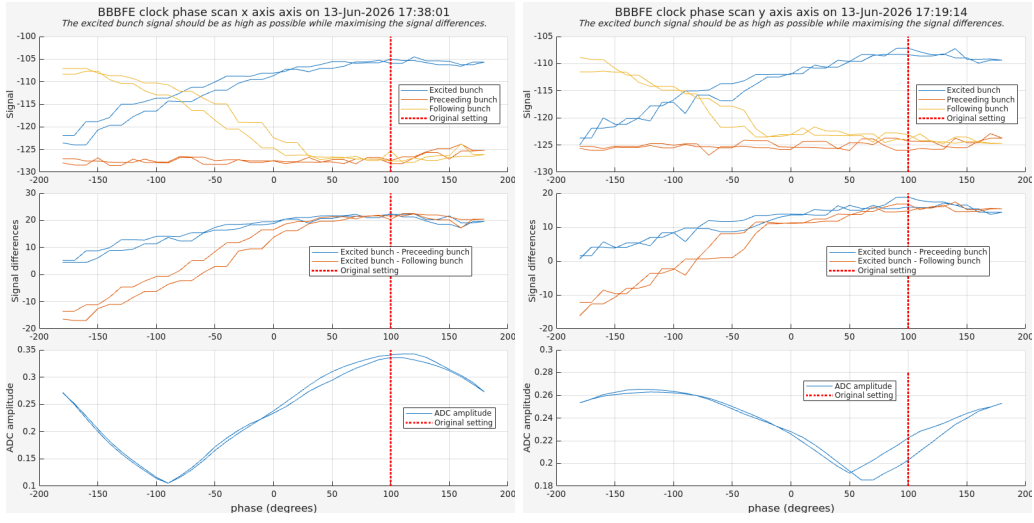
(a) Horizontal axis

(b) Vertical axis



(c) longitudinal axis

Figure 3.4: Example system phase results



(a) Horizontal axis

(b) Vertical axis



(c) longitudinal axis

Figure 3.5: Example clock phase results

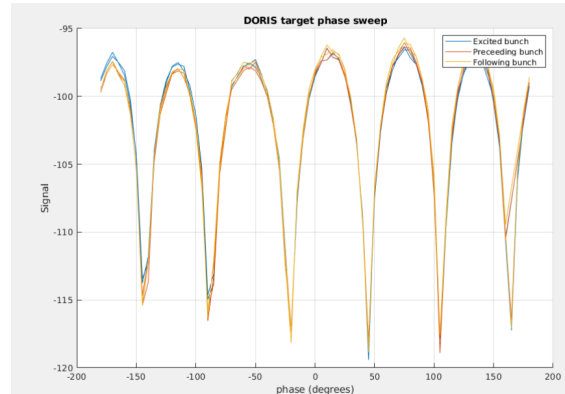


Figure 3.6: Example DORIS phase scan

3.3 Getting any historic data

A more flexible way is to set up the time period you are interested in looking at and use the measurement specific retrieval tool. If there is only one dataset in the chosen date range then the graphs in the previous section are created. Otherwise alternative graphs are plotted which better allow the investigation of variations over time or variation with current for example.

Again there are common settings

- Required

- `ax, {'x', 'y', 's'}`

Select the axis of interest.

- You can set up the period of interest by using a modified version the following example code.

```
>> start_datenum = datetime(2026, 04, 21, 15, 15, 00);
>> end_datenum = datetime(2026, 04, 21, 18, 00, 00);
>> date_range = [start_datenum, end_datenum];
```

- filter conditions. As machine conditions can be different it is possible to filter on metadata values. Usually current range and RF frequency range. This is often useful to ensure that any measurements used are below instability thresholds.

- Optional

- `'bypass_index', 'yes' or 'no'` Default is no

The codes write data to index files for faster retrieval. If the file is corrupted this option allows the retrieval code to use the underlying data.

- `'metadata_only', 'yes' or 'no'` Default is no

Useful to quickly find what measurement have been done.

- `'analysis_type', 'collate' or 'sweep'` Default is `'collate'`

Either just plot each dataset on top of one another, or sort them against a variable in the dataset.

- `'sweep_parameter', any variable in the dataset` Default is `'current'`

If analysis type is set to sweep this determines the variable to sweep against.

- `'parameter_step',` Default is 0.1

In the sweep datasets with a sweep value closer than this are averaged.

An example of putting this together

```
mbf_growdamp_archival_retrieval('y',
[datetime(2026, 04, 21, 15, 15, 00), datetime(2026, 04, 21, 18, 00, 00)],
{'current', [30 300]},
'analysis_type', 'sweep',
'sweep_parameter', 'current',
'parameter_step', 5)
```

In this case we are asking for all data between 3pm and 6pm on 21st April 2026, but only if the current is between 30mA and 300mA. We are asking the output to be plotted as a current sweep with a minimum step of 5mA.

3.3.1 Growdamp

The following command will extract any measurements taken within the date range for the y axis and then plot them on top of one another. An example of this is shown in figure 3.7.

```
>> mbf_growdamp_archival_retrieval('y', date_range, {});
```

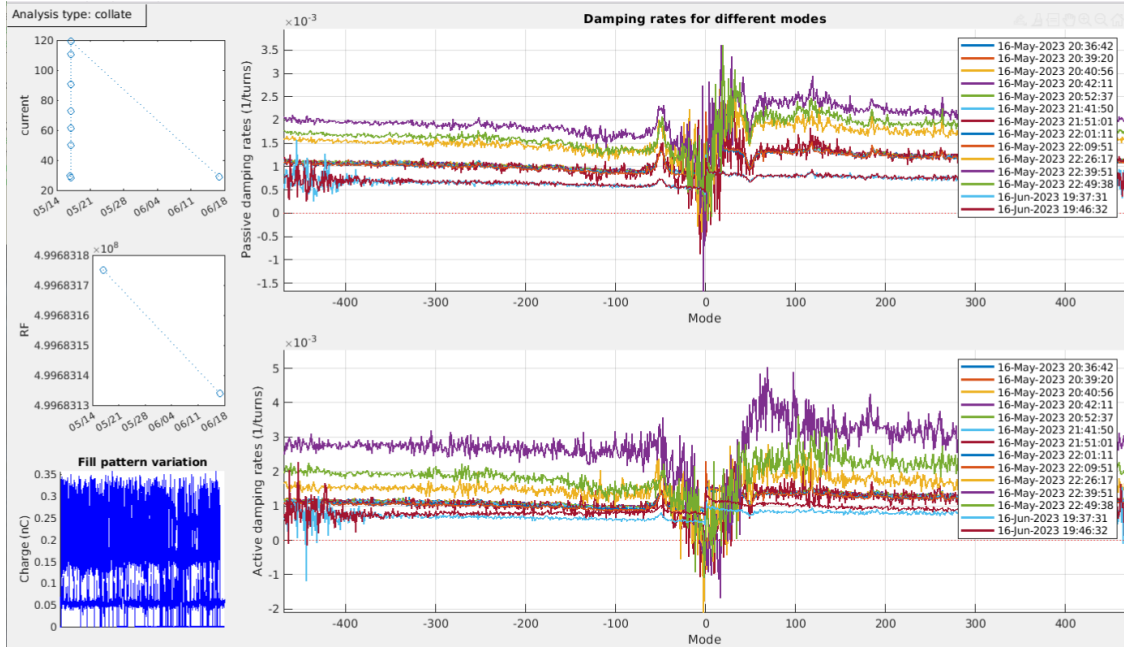


Figure 3.7: Collated growdamp results

This allows easy identification of changes in behaviour

Additional optional settings

- `'active_override'`, number of turns to analyse, Default is NaN for all
- `'passive_override'`, number of turns to analyse, Default is NaN for all
- `'advanced_fitting'`, allows alternative fitting algorithm to be used, Default is 0 for basic fitting
- `'debug'`, plots extra graphs to help identify the cause of odd behaviour, Default is 0 for off

Common filtering options

- `'current'`, [30 300] The current range to include
- `'excitation_setting'`, 'yes' or 'no'
This allows one to distinguish between experiments with driven growth and the machines natural growth.
- `'excitation_location'`, 'tune' or 'sideband' or 'manual'
This separates out different types of excitation.

Examples

One common set of measurements beyond simple comparisons is plotting the measurements against a changing machine parameter (usually current)

Although it is possible to extract such relationships from the data just using the variation over time, it is more usual to have a dedicated measurement set for the purpose. In this case you would set the period to only contain data from that experiment. Then you would run the retrieval with the analysis type set to sweep and then set the appropriate parameter to sweep over (again it is usually current but any variable in the datasets can be used). Figure 3.8 shows an example of this.

```
>> mbf_growdamp_archival_retrieval('y',...
[datetime(2026, 04, 21, 15, 15 ,00), datetime(2026, 04, 21, 18, 00, 00)],...
{'current', [30 300], 'excitation_setting', 'yes',...
'excitation_location', 'tune'}, ...
'analysis_type', 'sweep', 'parameter_step', 5)
```

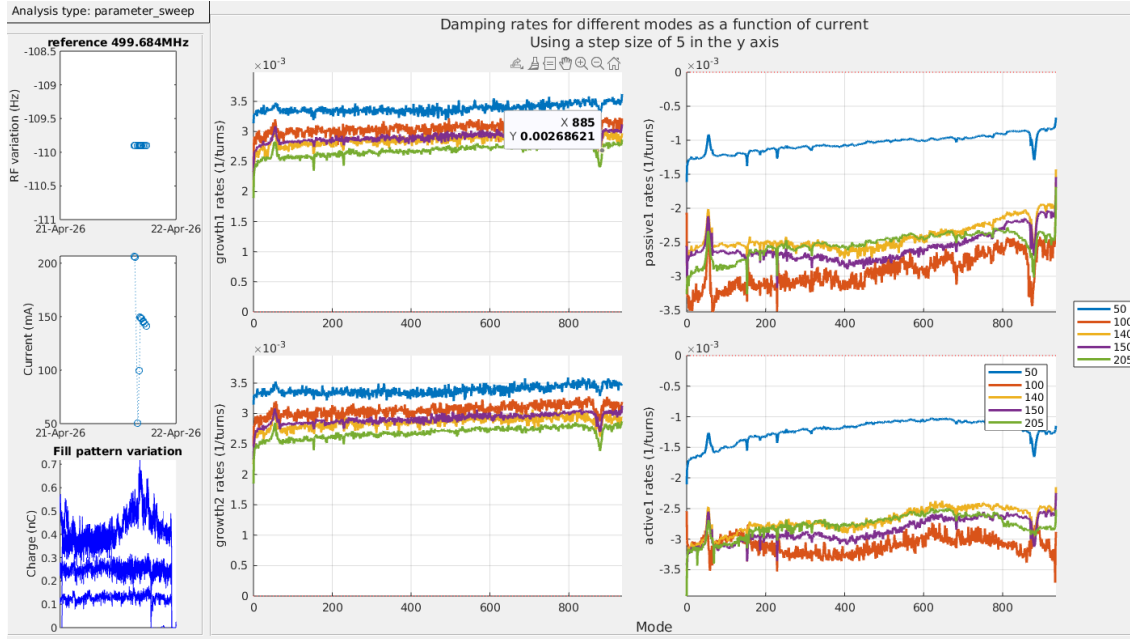


Figure 3.8: Growdamp results against a swept parameter. In this case the beam current. Note: These are placeholder graphs.

At this point further analysis is possible. In the case of a current sweep, it is possible to predict the current which individual modes will go unstable. (figures 3.9 and 3.10).

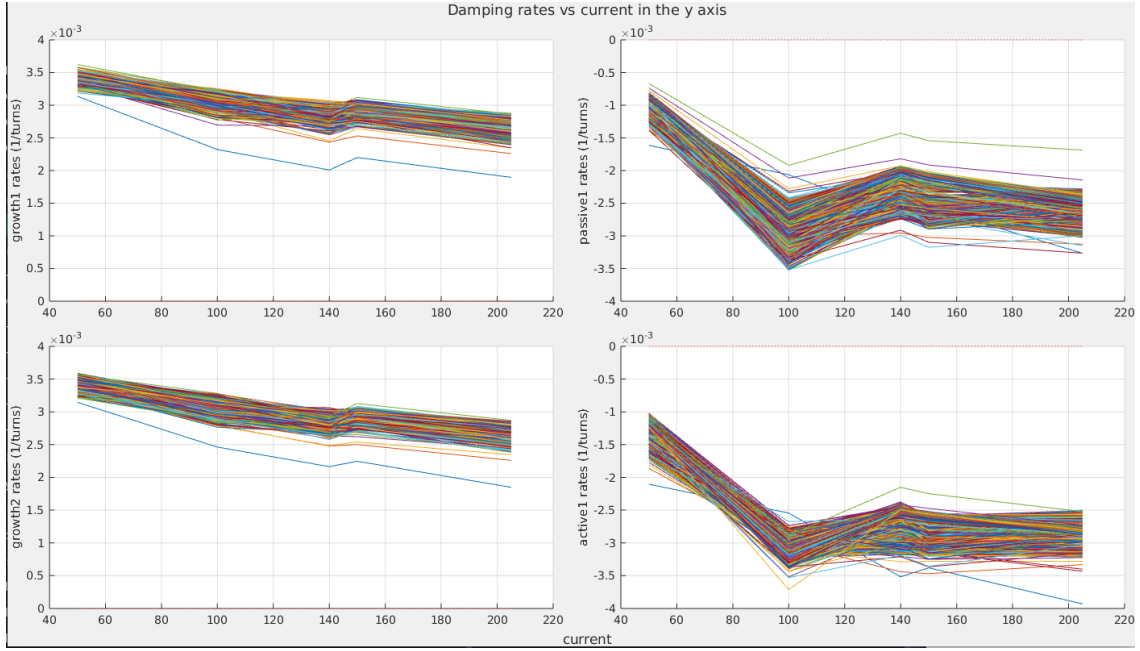


Figure 3.9: Results from further analysis of the current dependent growdamp data.

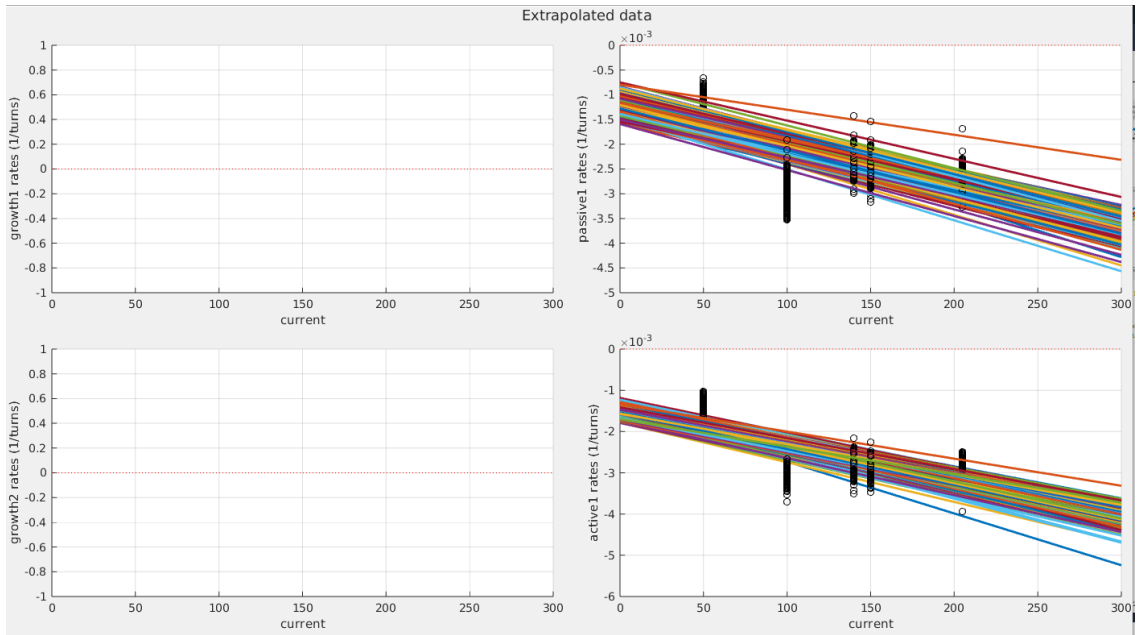


Figure 3.10: Results from further analysis of the current dependent growdamp data. Extrapolating to full operational current.

3.3.2 Modescan

Additional optional settings

- None

Common filtering options

- `'current'`, [30 300] The current range to include

Examples

Modescan data retrieval is much the same as the growdamp retrieval. Once the date range is set up the following code will extract and display all results from the y axis (figure 3.11).

```
>> mbf_modescan_archival_retrieval('y', ...
[datetime(2026, 04, 21, 15, 15, 00), datetime(2026, 04, 21, 18, 00, 00)],...
{'analysis_type', 'sweep', 'parameter_step', 5})
```

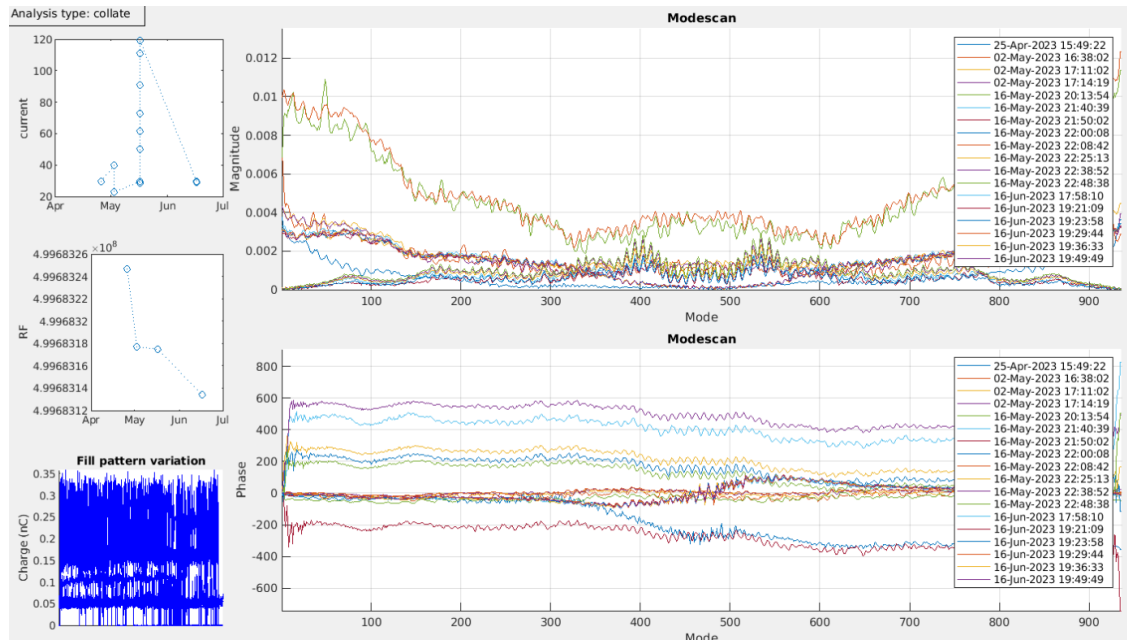


Figure 3.11: Collated modescan results

3.3.3 Spectra

Additional optional settings

Common filtering options

- `'current'`, [30 300] The current range to include

Examples

Tune sweep over modes